

EtherCAT Master & Slave 기술 설명서

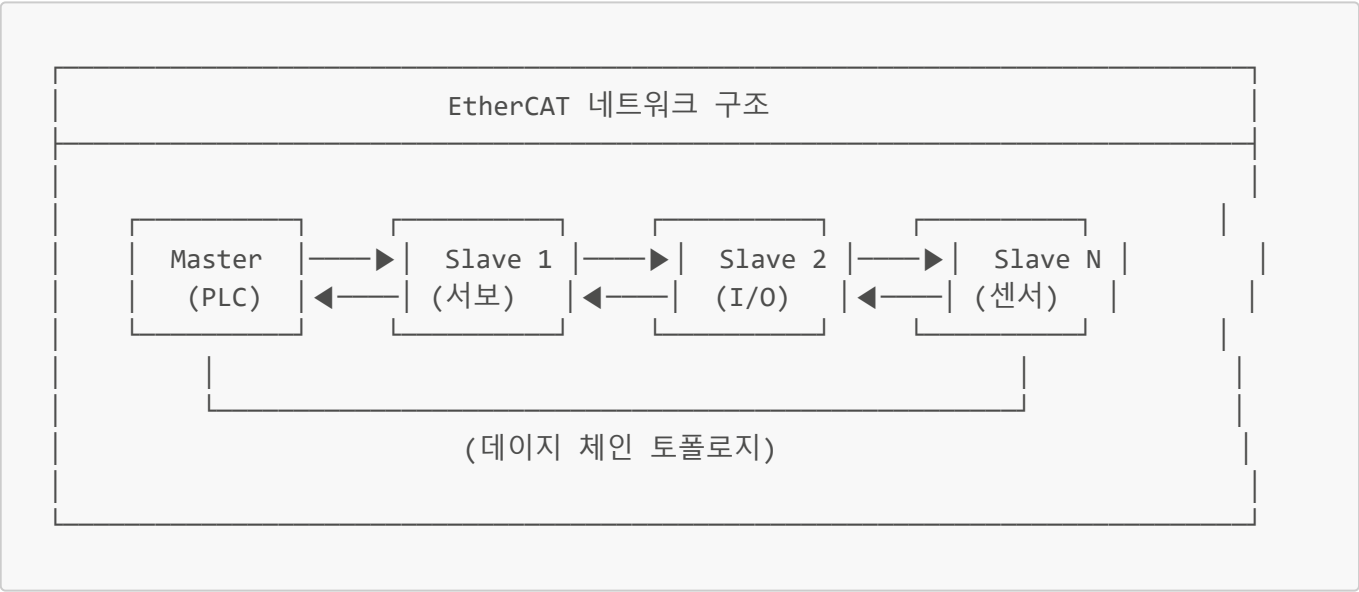
목차

- 1. EtherCAT 개요
- 2. EtherCAT Master
- 3. EtherCAT Slave
- 4. Master-Slave 통신 구조
- 5. 하드웨어 구현
- 6. 소프트웨어 스택
- 7. 개발 시 고려사항

1. EtherCAT 개요

1.1 EtherCAT이란?

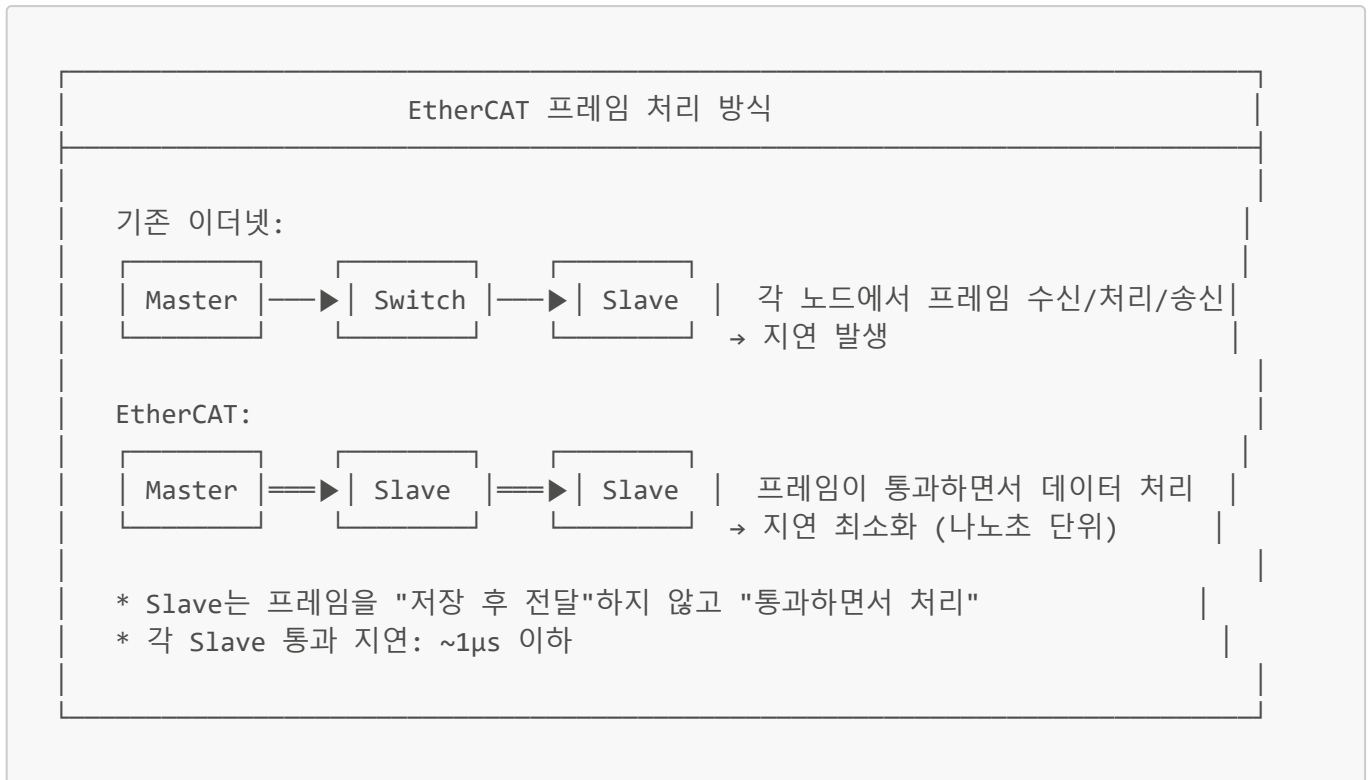
EtherCAT (Ethernet for Control Automation Technology)은 Beckhoff Automation에서 개발한 산업용 실시간 이더넷 프로토콜입니다.



1.2 핵심 특징

특징	설명
고속 통신	100Mbps 풀 듀플렉스
저지연	사이클 타임 <100μs 가능
정밀 동기화	분산 클럭(DC)으로 <1μs 동기화
유연한 토폴로지	라인, 트리, 스타 구조 지원
표준 이더넷	표준 이더넷 케이블/커넥터 사용

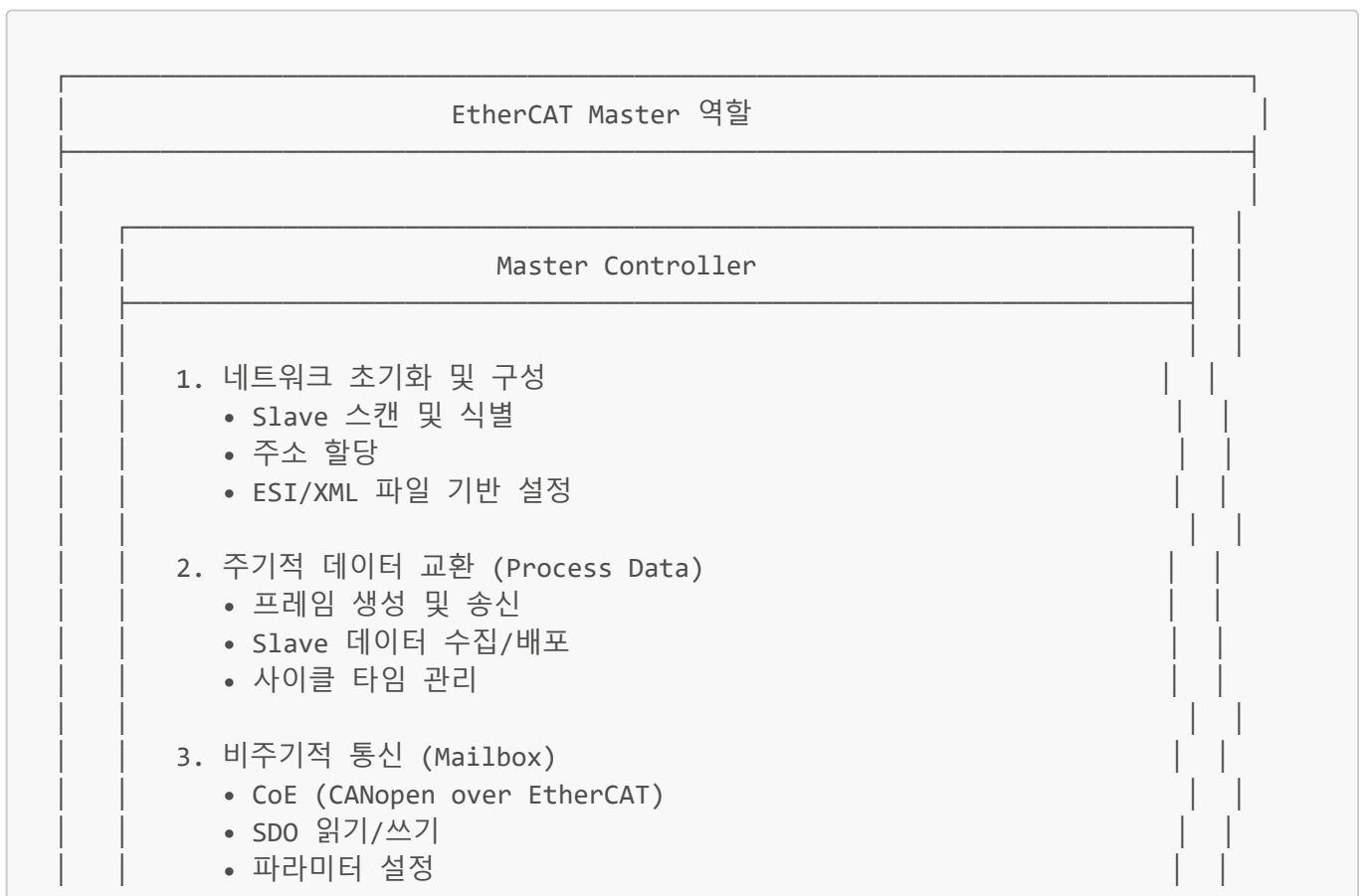
1.3 동작 원리: "On the Fly" 처리



2. EtherCAT Master

2.1 Master의 역할

EtherCAT Master는 네트워크의 **유일한 능동적 참여자**로서 모든 통신을 제어합니다.



4. 분산 클럭 동기화 (DC)

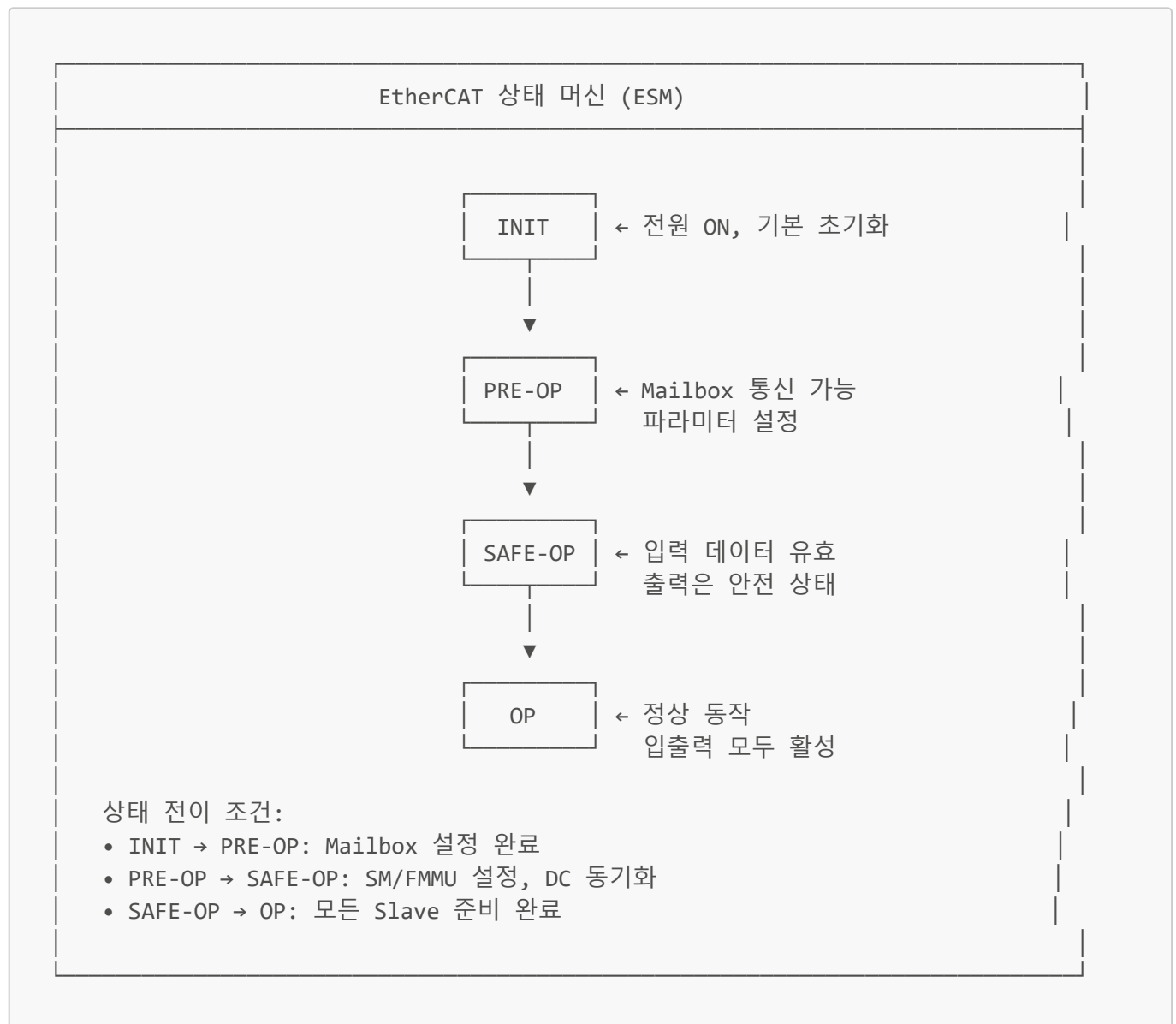
- 기준 클럭 선정
- 시간 오프셋 계산
- 동기 신호 관리

5. 진단 및 오류 처리

- 통신 상태 모니터링
- 오류 감지 및 복구
- 로그 기록

2.2 Master 상태 머신 (ESM - EtherCAT State Machine)

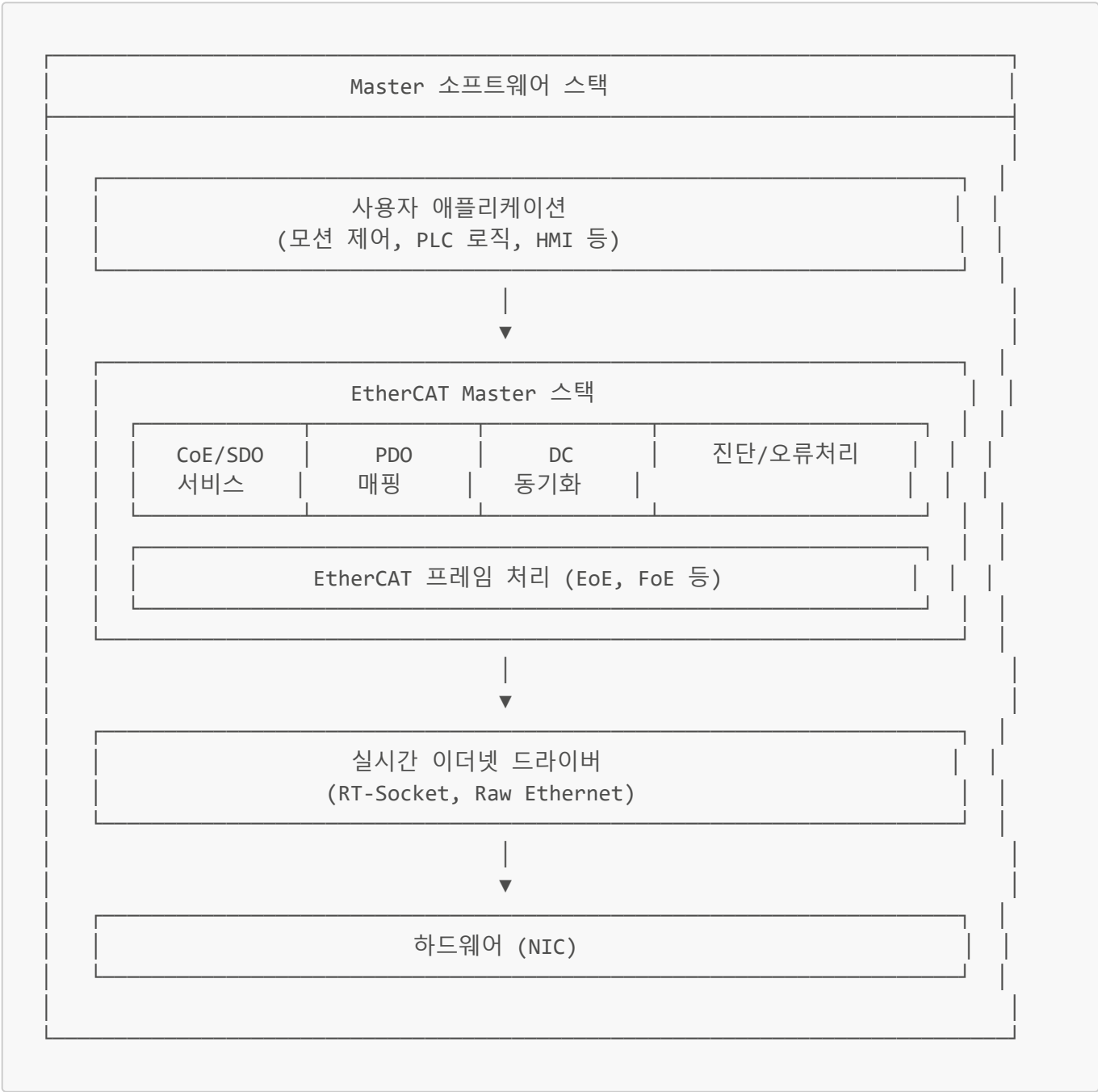
Master는 Slave들의 상태를 관리합니다:



2.3 Master 하드웨어 요구사항

구성요소	최소 요구사항	권장 사양
CPU	ARM Cortex-A7 이상	x86 멀티코어, ARM Cortex-A53+
RAM	256MB	1GB 이상
이더넷	100Mbps NIC	기가비트 NIC (실시간 패치)
OS	Linux (RT-PREEMPT)	Xenomai, RTAI, 또는 전용 RTOS
저장장치	4GB	16GB 이상 (로그 저장용)

2.4 Master 소프트웨어 스택



2.5 주요 오픈소스 Master 스택

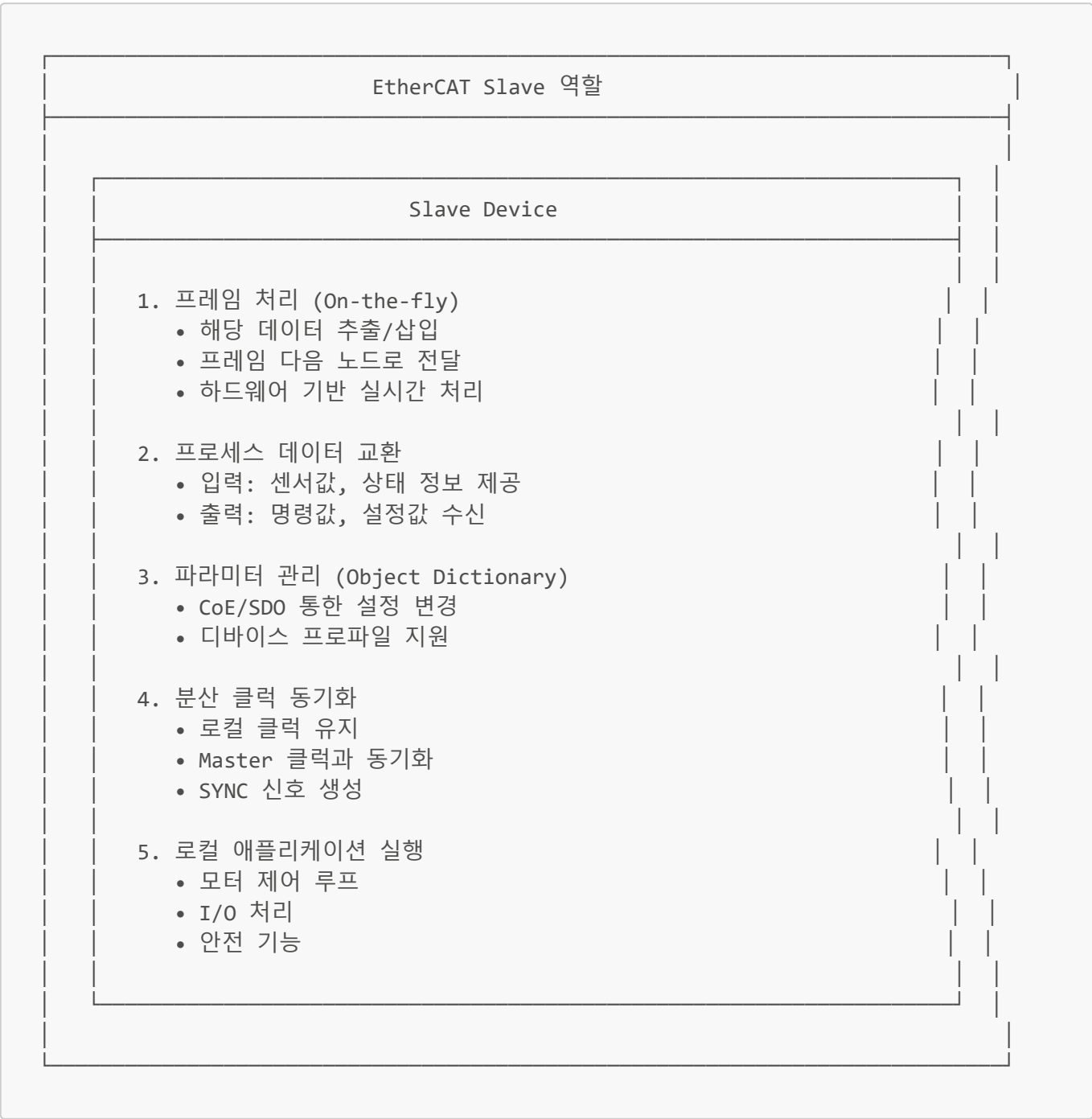
스택 이름	라이선스	특징
-------	------	----

스택 이름	라이선스	특징
SOEM	LGPL	경량, 임베디드 적합, C언어
IgH EtherCAT Master	GPL	리눅스 커널 모듈, 고성능
EtherLab	LGPL	MATLAB/Simulink 연동

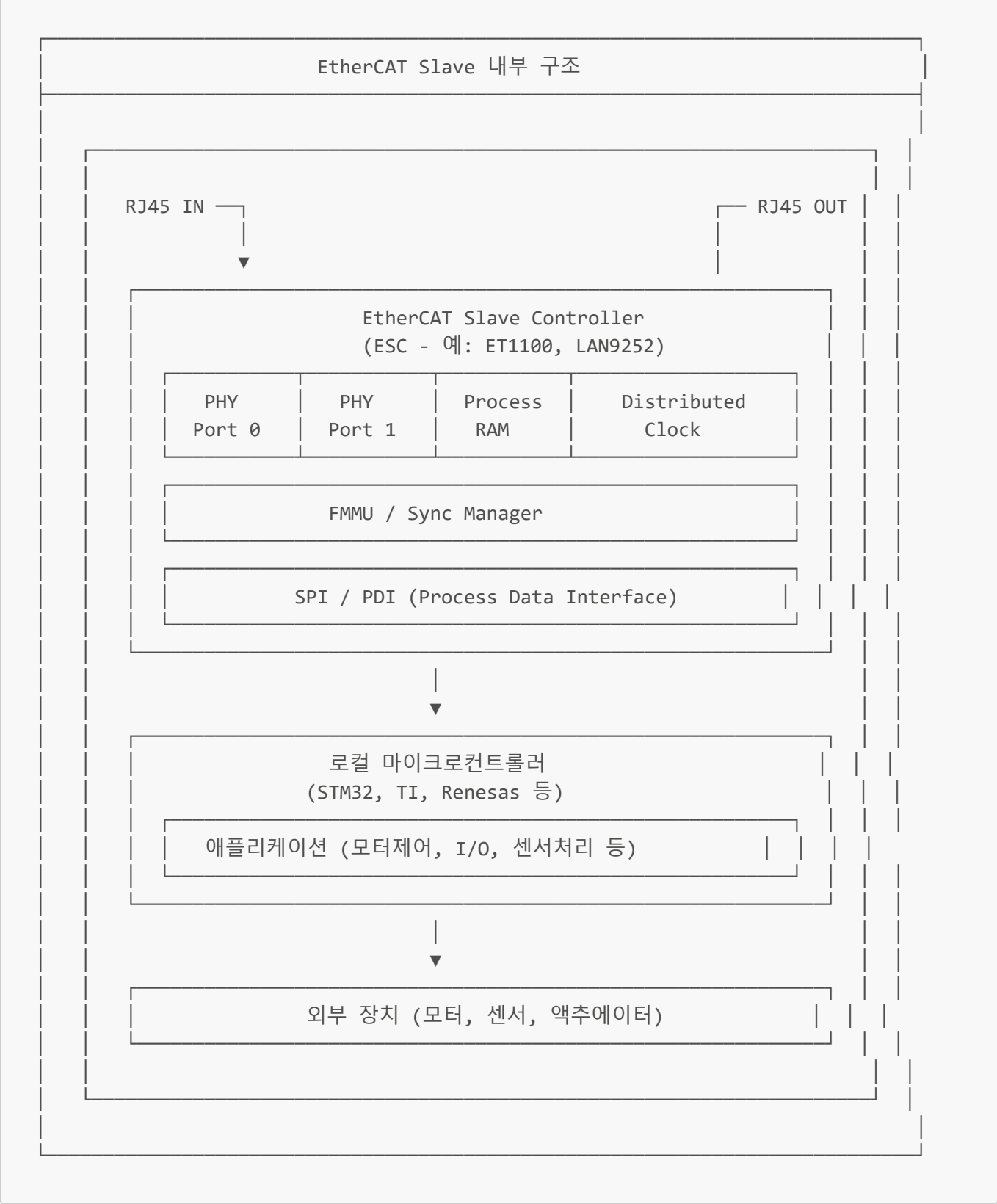
3. EtherCAT Slave

3.1 Slave의 역할

EtherCAT Slave는 네트워크의 **수동적 참여자**로서 Master의 명령에 응답합니다.



3.2 Slave 내부 구조

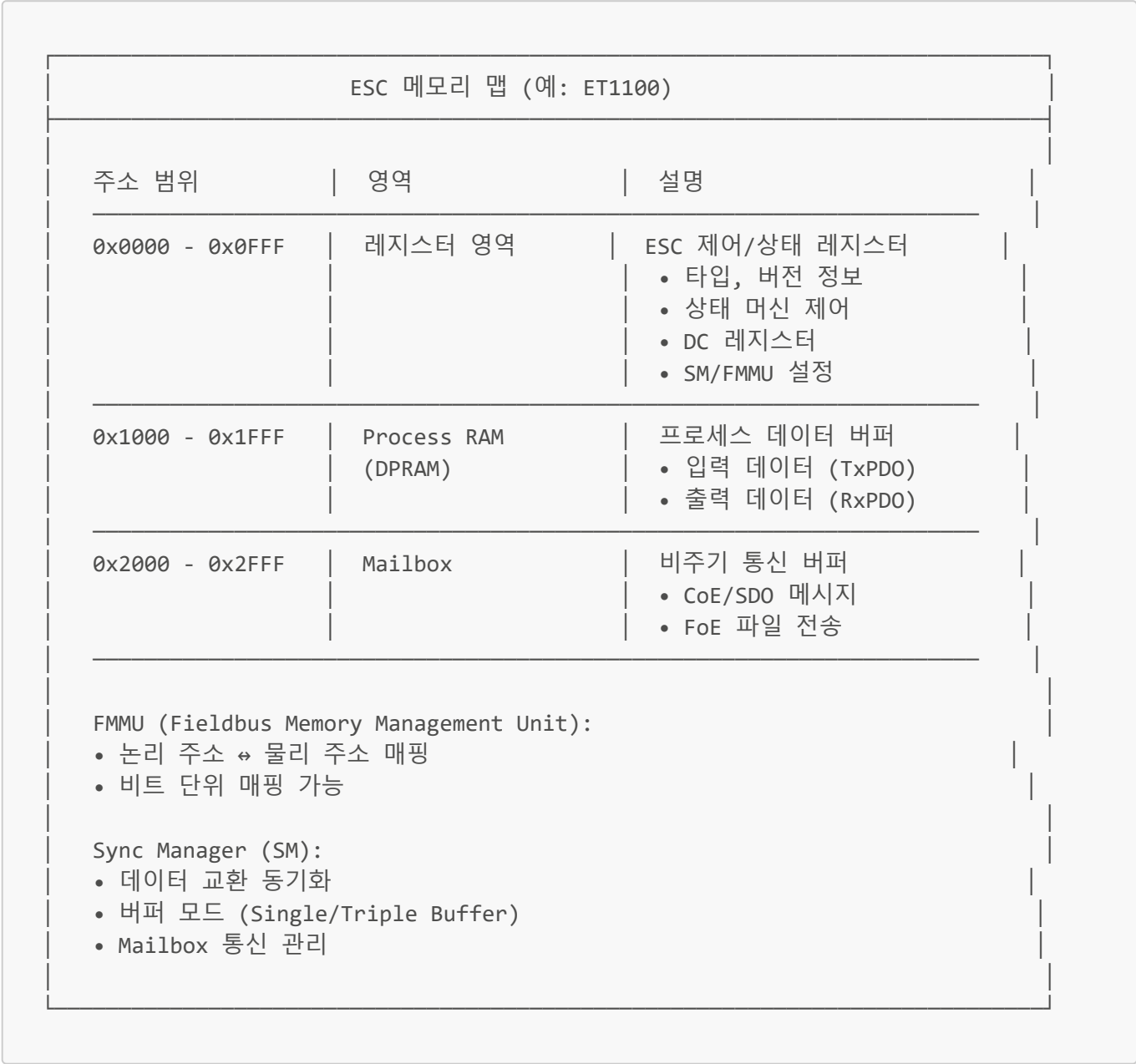


3.3 주요 ESC (EtherCAT Slave Controller) 칩

제조사	칩 모델	특징
Beckhoff	ET1100	표준 ESC, SPI/PDI 인터페이스
Beckhoff	ET1200	저비용, 소형
Microchip	LAN9252	통합 PHY, SPI/SQI

제조사	칩 모델	특징
Microchip	LAN9253	LAN9252 후속, 개선된 성능
TI	AMIC110	ARM Cortex-A8 + PRU 통합
Infineon	XMC4800	Cortex-M4 + EtherCAT 통합

3.4 Slave 메모리 구조



3.5 Slave 디바이스 프로파일

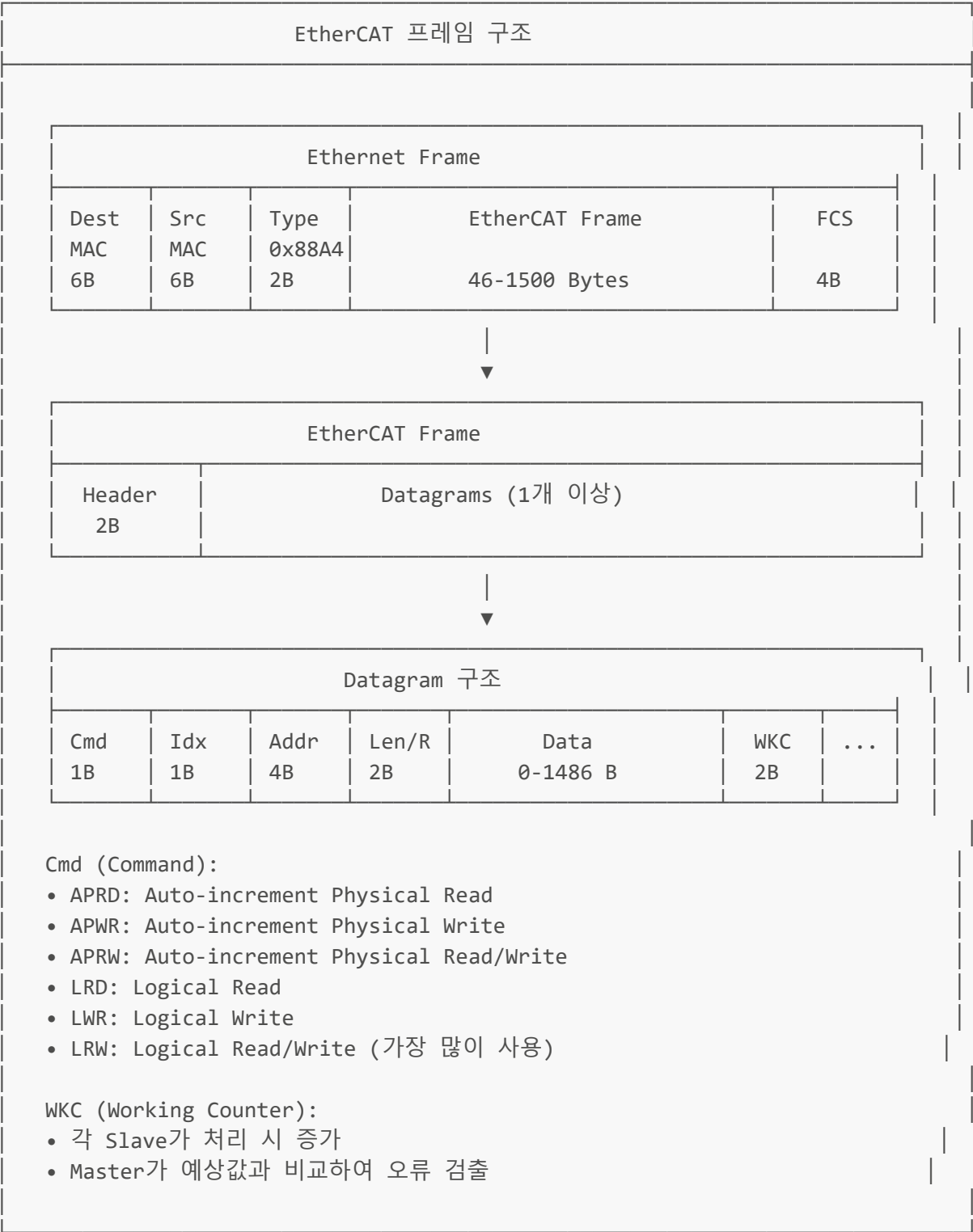
EtherCAT은 CANopen 기반의 디바이스 프로파일을 사용합니다:

프로파일	코드	적용 장치
CiA 402	DS402	서보 드라이브, 스테퍼
CiA 401	DS401	디지털/아날로그 I/O

프로파일	코드	적용 장치
CiA 406	DS406	엔코더
CiA 410	DS410	인클리노미터

4. Master-Slave 통신 구조

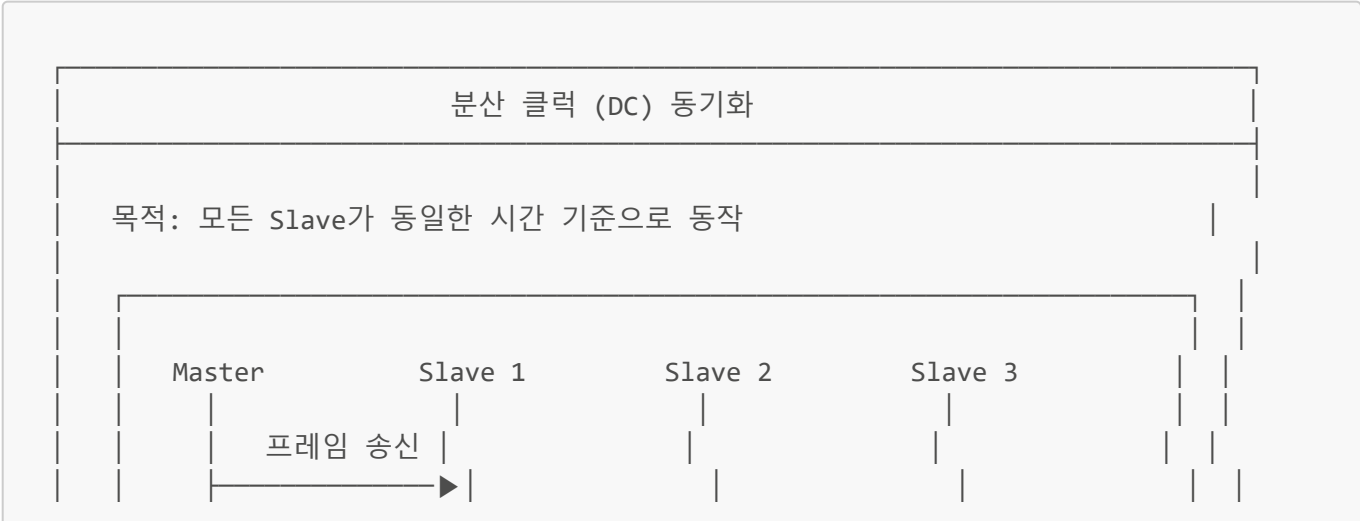
4.1 EtherCAT 프레임 구조

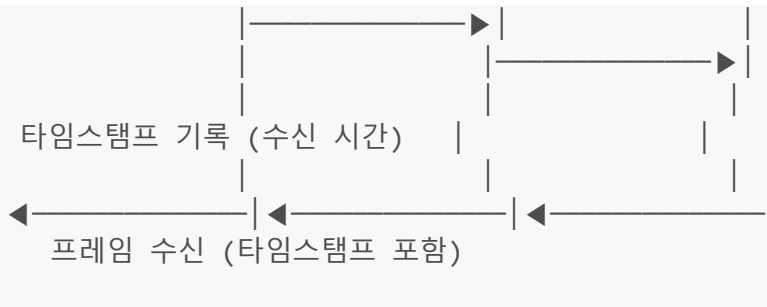


4.2 PDO (Process Data Object) 통신



4.3 분산 클럭 (Distributed Clock) 동기화





전파 지연 계산:

$$\text{delay} = (\text{수신시간} - \text{송신시간}) / 2$$

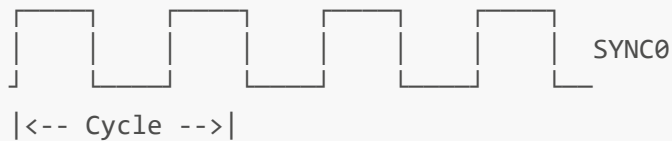
오프셋 보정:

각 Slave는 기준 클럭(보통 첫 번째 Slave)에 동기화

SYNC 신호:

SYNC0: 주기적 동기 신호 (예: 1ms 마다)
→ 입력 데이터 샘플링, 출력 데이터 적용

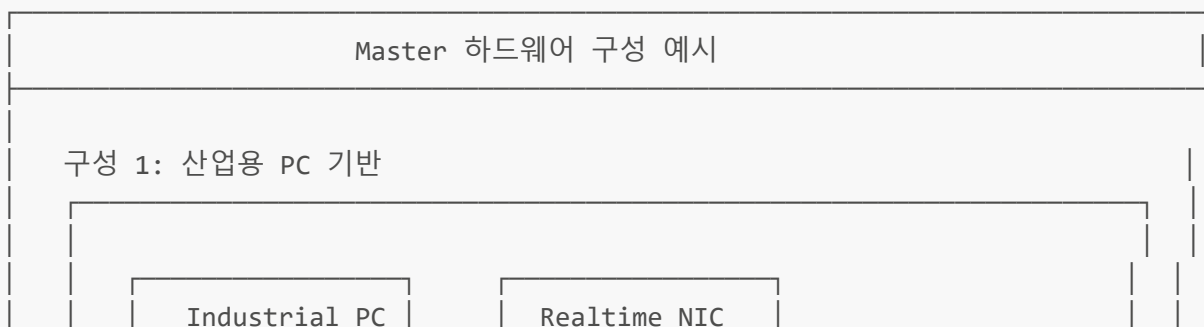
SYNC1: 보조 동기 신호 (SYNC0 이후 오프셋)
→ 추가 동기화 포인트

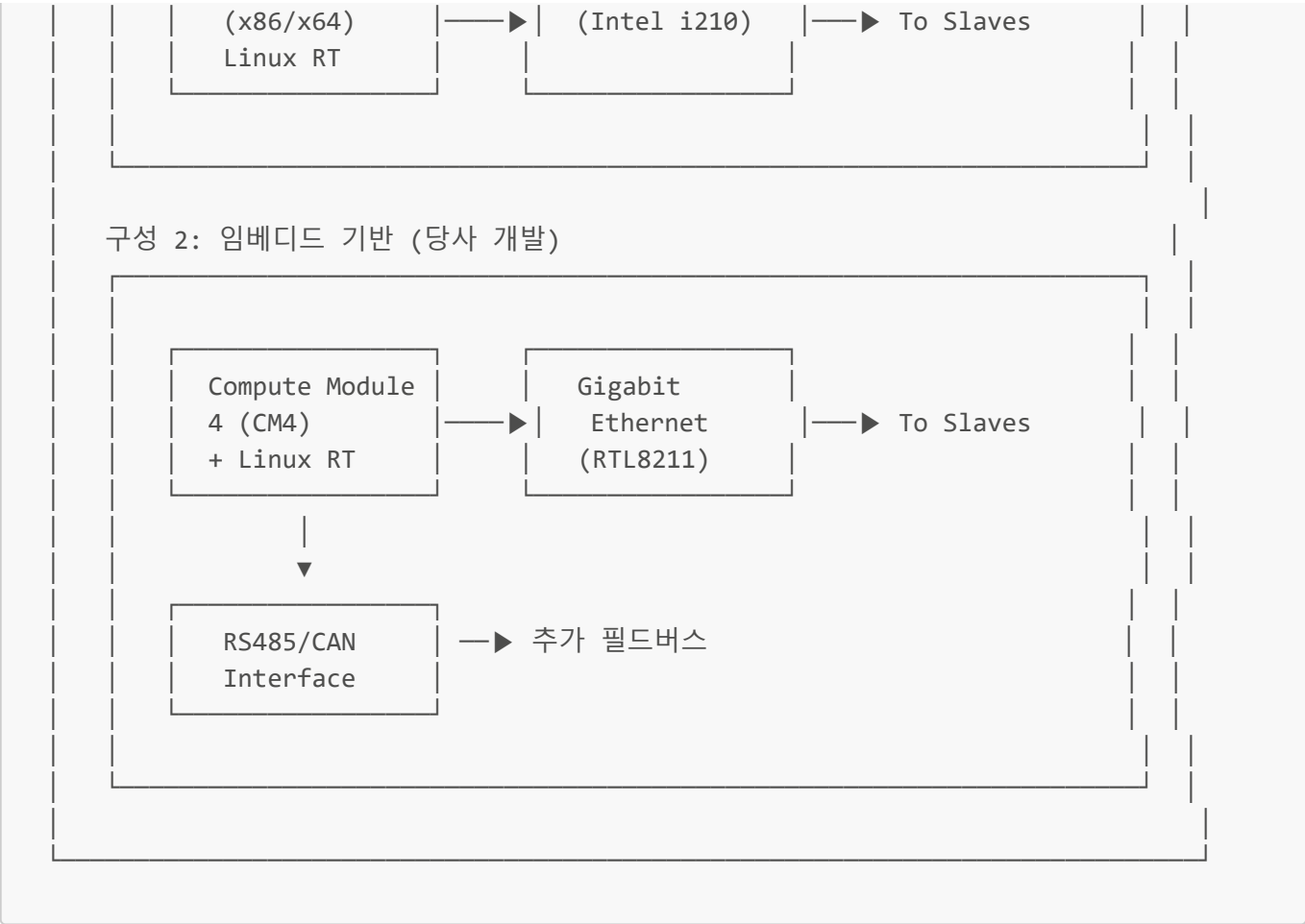


동기화 정밀도: < 1 μ s (일반적으로 100ns 이하)

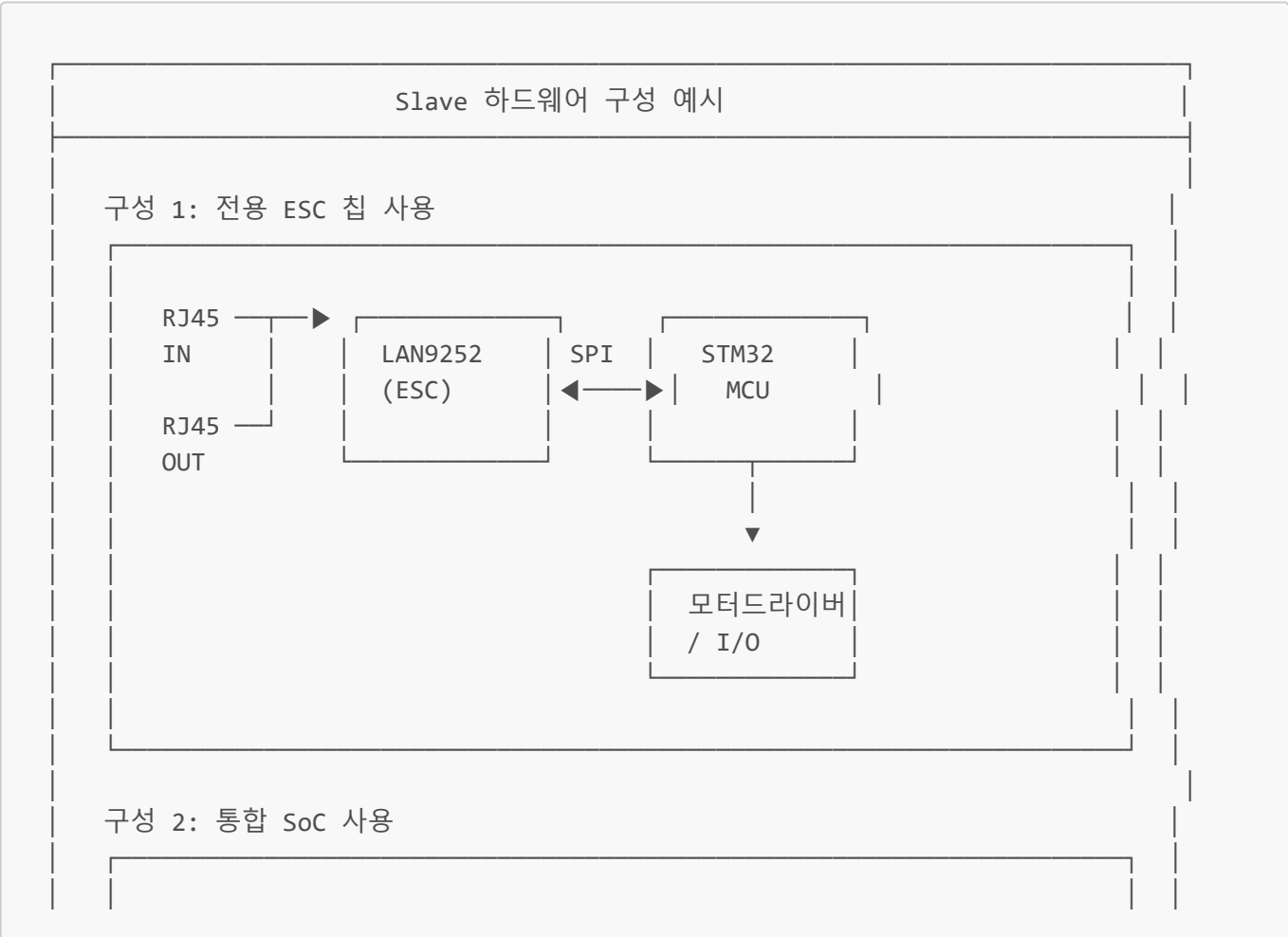
5. 하드웨어 구현

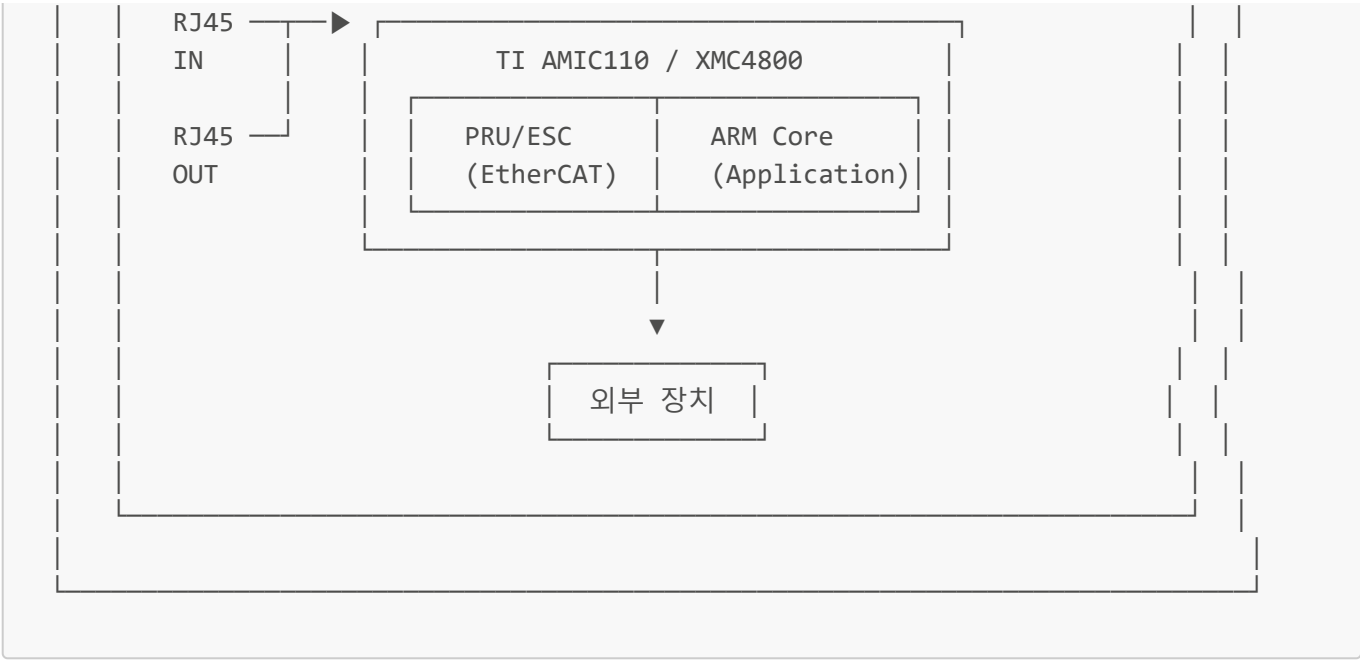
5.1 Master 하드웨어 구성





5.2 Slave 하드웨어 구성





5.3 회로 설계 시 고려사항

항목	고려사항
PHY 선택	100BASE-TX 지원, 저지연 PHY 권장
트랜스포머	1:1 절연 트랜스, 오토 MDI/MDIX
전원	ESC 전원 안정화, 노이즈 필터링
클럭	25MHz 크리스탈, 정밀도 ±50ppm 이하
커넥터	산업용 RJ45, M12 (IP67)
EMC	섀딩, 서지 보호, 접지 설계

6. 소프트웨어 스택

6.1 Master 소프트웨어 개발

```
/* SOEM (Simple Open EtherCAT Master) 예제 */

#include "ethercat.h"

char IOmap[4096];
int expectedWKC;

int main(int argc, char *argv[])
{
    /* 1. 네트워크 인터페이스 초기화 */
    if (ec_init("eth0") > 0)
    {
        printf("EtherCAT 초기화 성공\n");

        /* 2. Slave 스캔 및 설정 */
    }
}
```

```

if (ec_config_init(FALSE) > 0)
{
    printf("%d개의 Slave 발견\n", ec_slavecount);

    /* 3. PDO 매핑 및 DC 설정 */
    ec_config_map(&IOmap);
    ec_configdc();

    /* 4. 상태 전이: SAFE-OP → OP */
    ec_slave[0].state = EC_STATE_OPERATIONAL;
    ec_writestate(0);

    /* 5. 실시간 루프 */
    while(1)
    {
        /* PDO 교환 */
        ec_send_processdata();
        expectedWKC = ec_receive_processdata(EC_TIMEOUTRET);

        if (expectedWKC >= ec_group[0].expectedWKC)
        {
            /* 입력 데이터 처리 */
            process_inputs(IOmap);

            /* 출력 데이터 설정 */
            set_outputs(IOmap);
        }

        /* 사이클 타임 대기 (예: 1ms) */
        osal_usleep(1000);
    }
}

ec_close();
return 0;
}

```

6.2 Slave 소프트웨어 개발

```

/* SSC (Slave Stack Code) 기반 Slave 예제 */

#include "ecatslv.h"
#include "ecatappl.h"

/* Object Dictionary 정의 */
OBJCONST TOBJECT OBJMEM ApplicationObjDic[] = {
    /* Index 0x6000: 입력 데이터 */
    {0x6000, {DEFTYPE_UNSIGNED16, 0 | (OBJCODE_VAR << 8),
              "Input Data", &InputData, NULL, NULL}},

```

```

/* Index 0x7000: 출력 데이터 */
{0x7000, {DEFTYPE_UNSIGNED16, 0 | (OBJCODE_VAR << 8),
          "Output Data", &OutputData, NULL, NULL}},
{0xFFFF, {0, 0, NULL, NULL, NULL, NULL}}
};

/* 메인 애플리케이션 */
void APPL_Application(void)
{
    /* ESC에서 출력 데이터 읽기 */
    HW_EscReadWord(OutputData, OUTPUTS_ADDRESS);

    /* 로컬 처리 (모터 제어 등) */
    ProcessLocalApplication();

    /* 입력 데이터 ESC에 쓰기 */
    HW_EscWriteWord(InputData, INPUTS_ADDRESS);
}

/* DC SYNC0 인터럽트 핸들러 */
void APPL_Sync0Event(void)
{
    /* 동기화된 데이터 처리 */
    /* 정확한 타이밍에 출력 적용 */
    ApplyOutputs();

    /* 입력 샘플링 */
    SampleInputs();
}

/* 상태 전이 콜백 */
UINT16 APPL_StartMailboxHandler(void)
{
    return 0; /* 성공 */
}

UINT16 APPL_StopMailboxHandler(void)
{
    return 0;
}

UINT16 APPL_StartInputHandler(UINT16 *pIntMask)
{
    /* DC 인터럽트 활성화 */
    *pIntMask = 0x0004; /* SYNC0 인터럽트 */
    return 0;
}

```

7. 개발 시 고려사항

7.1 성능 최적화

성능 최적화 가이드

Master 최적화:

1. 실시간 커널 사용 (RT-PREEMPT, Xenomai)

2. CPU 코어 격리 (isolcpus)

3. 네트워크 인터럽트 튜닝

4. 메모리 락 (mlockall)

5. 실시간 우선순위 설정

Slave 최적화:

1. DMA 활용 (SPI/PDI 통신)

2. 인터럽트 지연 최소화

3. PDO 크기 최적화

4. 불필요한 기능 비활성화

네트워크 최적화:

1. 점보 프레임 비활성화

2. 플로우 컨트롤 비활성화

3. 케이블 품질 확보 (Cat5e 이상)

4. 적절한 토폴로지 설계

7.2 디버깅 및 진단

도구	용도
Wireshark	EtherCAT 프레임 분석 (ethercat 필터)
TwinCAT	Beckhoff 개발 환경, Slave 설정
ethercat tool	IgH Master 커맨드라인 도구
오실로스코프	타이밍, SYNC 신호 분석

7.3 인증 및 규격

인증	설명
ETG.5000	EtherCAT Slave Implementation Guide
ETG.1000	EtherCAT Specification
Conformance Test	ETG 공식 적합성 테스트
CE/UL	제품 안전 인증

부록: 용어 정리

용어	설명
ESC	EtherCAT Slave Controller
ESI	EtherCAT Slave Information (XML 파일)
FMMU	Fieldbus Memory Management Unit
SM	Sync Manager
PDO	Process Data Object
SDO	Service Data Object
CoE	CANopen over EtherCAT
DC	Distributed Clock
WKC	Working Counter
LRW	Logical Read/Write

작성일: 2026년 2월 14일 (주)OOO 기술팀